

Εργασία στα Δίκτυα Υπολογιστών Ι - source code

Σπυρίδων Μπάλτσας

AEM: 10443

[2023-05-10 Wed 01:01]

1 Περίληψη

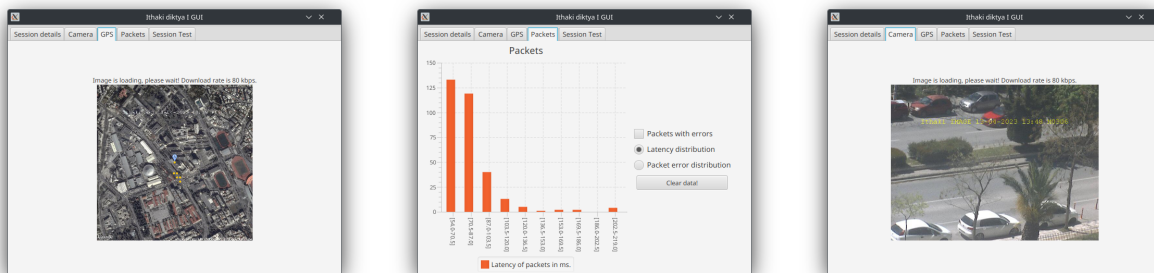


Figure 1: Στιγμιότυπα από την εφαρμογή.

Το παρόν έγγραφο περιέχει τον πηγαίο κώδικα (source code) της εργασίας του μαθήματος “Δίκτυα Υπολογιστών Ι”. Η εφαρμογή αναπτύχθηκε σε Java, χρησιμοποιώντας τον Eclipse IDE, καθώς και το maven για την διαχείριση των εξωτερικών βιβλιοθηκών και την προσθήκη του απαραίτητου για την εργασία *ithakimodem.jar*. Επίσης, χρησιμοποιήθηκε η βιβλιοθήκη JavaFX για την ανάπτυξη των γραφικών. Οι δυνατότητες της εφαρμογής είναι οι ακόλουθες:

- Αυτόματη λήψη των κωδικών του session απο την ιθάκη.
- Ζωντανή, κατά το δυνατό, μετάδοση απο την σταθερή κάμερα.
- Λήψη στιγμάτων από το GPS.
- Κατανομή των πακέτων συναρτήσει της καθυστέρησης σε πραγματικό χρόνο.
- Εξαγωγή των αποτελεσμάτων των echo tests / ARQ tests σε μορφή CSV για εύκολη περαιτέρω στατιστική επεξεργασία.

2 Πηγαίος κώδικας (source code)

2.1 Δομή του φακέλου της εφαρμογής

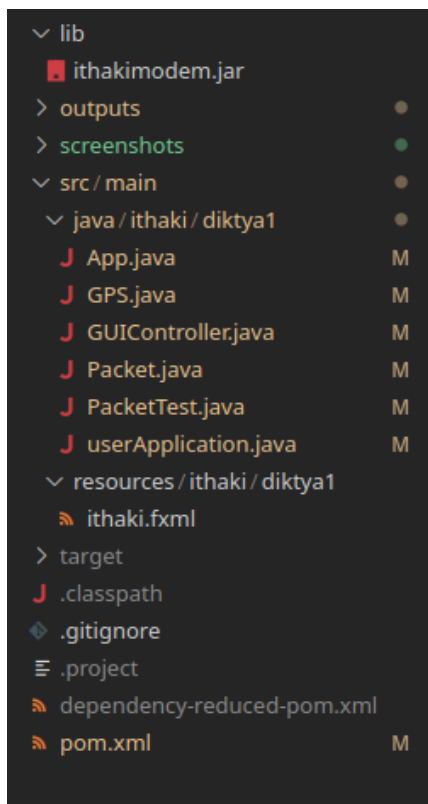


Figure 2: Φωτογραφία της δομής των αρχείων

2.2 Μεταγλώττιση και εγκατάσταση βιβλιοθηκών

Για να εγκατασταθούν οι βιβλιοθήκες του Maven (pom.xml), πρέπει να εκτελεστούν οι ακόλουθες εντολές:

1. `mvn clean`, για να εγκατασταθεί στο project η βιβλιοθήκη *ithakimodem.jar*
2. `mvn clean javafx:run`, για να εγκατασταθούν στο project οι υπόλοιπες βιβλιοθήκες και να μεταγλωττιστεί το πρόγραμμα.

Τέλος, για να δημιουργηθεί το jar, αφού εκτελεστούν όλα τα παραπάνω βήματα, αρκεί η εντολή `mvn install`.

2.3 Κώδικας Java

2.3.1 userApplication.java

```
1 package ithaki.diktya1;
2
3
4 public class userApplication {
5
6     public static void main(String[] args) {
7         // TODO Auto-generated method stub
8         App.main(args);
9     }
10 }
11
```

2.3.2 App.java

```
1 package ithaki.diktya1;
2
3 import javafx.application.Application;
4 import javafx.event.EventHandler;
5 import javafx.fxml.FXMLLoader;
6 import javafx.scene.Parent;
7 import javafx.scene.Scene;
8 import javafx.stage.Stage;
9 import javafx.stage.WindowEvent;
10
11 import java.io.IOException;
12 import java.net.URI;
13 import java.net.http.HttpClient;
14 import java.net.http.HttpRequest;
15 import java.net.http.HttpResponse;
16 import java.net.http.HttpClient.Redirect;
17 import java.net.http.HttpClient.Version;
18 import java.nio.file.Files;
19 import java.nio.file.Paths;
20 import java.time.Duration;
21 import java.util.HashMap;
22 import java.util.regex.Matcher;
23 import java.util.regex.Pattern;
24
25 import org.jsoup.Jsoup;
26 import org.jsoup.nodes.Document;
27 import org.jsoup.nodes.Element;
28
29 /**
30  * JavaFX App
31  */
32 public class App extends Application {
33
34     protected static HashMap<String, String> CODES = new HashMap<>();
35
36     private static Scene scene;
```

```

37
38  @Override
39  public void start(Stage stage) throws IOException {
40      getSessionCodes();
41      try {
42          Files.createDirectories(Paths.get("outputs/images"));
43          Files.createDirectories(Paths.get("outputs/csv"));
44      } catch (IOException e) {
45          // TODO Auto-generated catch block
46          e.printStackTrace();
47      }
48      scene = new Scene(loadFXML("ithaki"), 640, 480);
49      stage.setTitle("Ithaki diktya I GUI");
50      stage.setScene(scene);
51      stage.setResizable(false);
52      stage.setOnCloseRequest(new EventHandler<WindowEvent>() {
53
54          @Override
55          public void handle(WindowEvent event) {
56              // TODO Auto-generated method stub
57              //GUIController.killAllThreads = true;
58              System.exit(0);
59          }
60      });
61      stage.show();
62  }
63
64  /*static void setRoot(String fxml) throws IOException {
65      scene.setRoot(loadFXML(fxml));
66  }*/
67
68  private static Parent loadFXML(String fxml) throws IOException {
69      FXMLLoader fxmlLoader = new FXMLLoader(App.class.getResource(fxml + ".fxml"));
70      return fxmlLoader.load();
71  }
72
73  public static void main(String[] args) {
74      launch();
75  }
76
77  private static void getSessionCodes() {
78      try {
79
80          HttpClient client = HttpClient.newBuilder()
81              .version(Version.HTTP_2)
82              .connectTimeout(Duration.ofSeconds(30))
83              .followRedirects(Redirect.NORMAL)
84              .build();
85          HttpRequest request = HttpRequest.newBuilder()
86              .uri(URI.create("http://ithaki.eng.auth.gr/netlab/main.php"))
87              .setHeader("Content-Type", "application/x-www-form-urlencoded")
88              .setHeader("User-Agent", "Mozilla/5.0 (X11; Linux x86_64)
89                  ↪ AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.114
90                  ↪ Safari/537.36")
91              .POST(HttpRequest.BodyPublishers.ofString("fi=ΣΠΤΡΙΑΔΩΝ&fa=ΜΠΑΛΤΑΣ&am=1044_
92                  ↪ 3&op=1"))

```

```

90         .build();
91     HttpResponse<String> response = client.send(request,
92         ↪ HttpResponse.BodyHandlers.ofString());
93     Document document = Jsoup.parse(response.body());
94     Element sessionElement = document.select("input[name=session]").first();
95     String sessionIDString = sessionElement.attr("value");
96     System.out.println("session id "+sessionIDString);
97     HttpRequest sessionCodesRequest = HttpRequest.newBuilder()
98         .uri(URI.create("http://ithaki.eng.auth.gr/netlab/action.php"))
99         .setHeader("Content-Type", "application/x-www-form-urlencoded")
100         .setHeader("User-Agent", "Mozilla/5.0 (X11; Linux x86_64)
101         ↪ AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.114
102         ↪ Safari/537.36")
103         .POST(HttpRequest.BodyPublishers.ofString("fi=ΣΠΤΡΙΑΩΝ&fa=ΜΠΑΛΤΕΑΣ&am=1044
104         ↪ 3&x=1&session="+sessionIDString))
105         .build();
106     HttpResponse<String> codesHTML = client.send(sessionCodesRequest,
107         ↪ HttpResponse.BodyHandlers.ofString());
108     //System.out.println(codesHTML.body());
109     Document codesDocument = Jsoup.parse(codesHTML.body());
110     //select only the specific table element, so regex will not have to search the
111     ↪ whole html all over again.
112     Element tableElement = codesDocument.select("td[align=left]").get(1);
113     Matcher matcher = Pattern.compile("E\\d+|M\\d+|G\\d+|P\\d+|Q+\\d+|R\\d+").matcher(
114     ↪ tableElement.text());
115     String match;
116     while(matcher.find()) {
117         match = matcher.group();
118         if(match.startsWith("E")) {
119             CODES.put("ECHO", match);
120         }
121         else if(match.startsWith("M")) {
122             CODES.put("IMAGE_NO_ERROR", match);
123         }
124         else if(match.startsWith("G")) {
125             CODES.put("IMAGE_ERROR", match);
126         }
127         else if(match.startsWith("P")) {
128             CODES.put("GPS", match);
129         }
130         else if(match.startsWith("Q")) {
131             CODES.put("ACK", match);
132         }
133         else {
134             CODES.put("NACK", match);
135         }
136     }
137     System.out.println(CODES.toString());
138
139 } catch (InterruptedException e) {
140     // TODO: handle exception
141     e.printStackTrace();
142 }
143
144 catch (IOException e) {
145     // TODO: handle exception
146     e.printStackTrace();
147 }

```

```
139     }  
140 }  
141  
142  
143 }
```

2.3.3 GUIController.java

```
1  package ithaki.diktya1;  
2  
3  import java.io.BufferedReader;  
4  import java.io.ByteArrayInputStream;  
5  import java.io.File;  
6  import java.io.FileNotFoundException;  
7  import java.io.FileOutputStream;  
8  import java.io.IOException;  
9  import java.io.StringReader;  
10 import java.util.ArrayList;  
11  
12 import ithakimodem.Modem;  
13  
14 import javafx.application.Platform;  
15 import javafx.beans.value.ChangeListener;  
16 import javafx.beans.value.ObservableValue;  
17 import javafx.collections.FXCollections;  
18 import javafx.fxml.FXML;  
19 import javafx.scene.control.Tab;  
20 import javafx.scene.control.TabPane;  
21 import javafx.scene.control.TextArea;  
22 import javafx.scene.control.TextFormatter;  
23 import javafx.scene.image.Image;  
24 import javafx.scene.image.ImageView;  
25 import javafx.scene.chart.BarChart;  
26 import javafx.scene.chart.XYChart;  
27 import javafx.scene.control.Alert;  
28 import javafx.scene.control.Alert.AlertType;  
29 import javafx.scene.control.Button;  
30 import javafx.scene.control.ButtonType;  
31 import javafx.scene.control.CheckBox;  
32 import javafx.scene.control.Label;  
33 import javafx.scene.control.RadioButton;  
34 import javafx.scene.control.Spinner;  
35 import javafx.scene.control.SpinnerValueFactory;  
36  
37 public class GUIController {  
38     protected enum DataTypes{PACKETS, GPS, INITIALIZATION};  
39     protected static volatile boolean stopSessionTest = false;  
40     private static final int MODEM_BITRATE = 8000;  
41     private final int MODEM_TIMEOUT = 3000;  
42     private boolean latencyDistribution = true;  
43     private ArrayList<Packet> packets;  
44     private Thread cameraThread, packetThread, gpsThread, sessionTestThread;  
45     private Tab selectedTab;  
46     private int min, max, range;  
47 }
```

```

48  @FXML
49  private TabPane tabPane;
50  @FXML
51  private ImageView cameraView, gpsView ;
52  @FXML
53  private Tab sessTab, cameraTab, gpsTab, packetsTab, testTab;
54  @FXML
55  private Label echoLabel, imageLabel, imageErrorLabel, gpsLabel, ackLabel, nackLabel;
56  @FXML
57  private TextArea testTextArea;
58  @FXML
59  private Button sessionTestBtn;
60  @FXML
61  private BarChart<String, Number> packetChart;
62  @FXML
63  private CheckBox errorPackets;
64  @FXML
65  private RadioButton latencyDist, packerrDist;
66  @FXML
67  private Spinner<Integer> packetdurationSpinner, imageCountSpinner, imageErrorCountSpinner;
68
69  @FXML
70  public void initialize() {
71      echoLabel.setText(App.CODES.get("ECHO"));
72      imageLabel.setText(App.CODES.get("IMAGE_NO_ERROR"));
73      imageErrorLabel.setText(App.CODES.get("IMAGE_ERROR"));
74      gpsLabel.setText(App.CODES.get("GPS"));
75      ackLabel.setText(App.CODES.get("ACK"));
76      nackLabel.setText(App.CODES.get("NACK"));
77      packetChart.setAnimated(false);
78      packetdurationSpinner.setValueFactory(new
79      ↪ SpinnerValueFactory.IntegerSpinnerValueFactory(1,86400));
80      imageCountSpinner.setValueFactory(new
81      ↪ SpinnerValueFactory.IntegerSpinnerValueFactory(1, 10));
82      imageErrorCountSpinner.setValueFactory(new
83      ↪ SpinnerValueFactory.IntegerSpinnerValueFactory(1, 10));
84      Runnable cameraRunnable = new Runnable() {
85          @Override
86          public void run() {
87              // TODO Auto-generated method stub
88              try {
89                  //wait for all other threads to finish first
90                  if(gpsThread != null) {
91                      if(gpsThread.isAlive()) gpsThread.join();
92                  }
93                  if(packetThread != null) {
94                      if(packetThread.isAlive()) packetThread.join();
95                  }
96                  if(sessionTestThread != null) {
97                      if(sessionTestThread.isAlive()) sessionTestThread.join();
98                  }
99                  Modem modem = new Modem(MODEM_BITRATE);
100                 modem.setTimeout(MODEM_TIMEOUT);
101                 getText(modem,null,"ithaki", DataTypes.INITIALIZATION);
102                 while(cameraTab.isSelected()) {
103                     byte[] image = downloadImage(modem, true, null);

```

```

101         System.out.println(image);
102         Platform.runLater(()->{
103             Image frame = new Image(new ByteArrayInputStream(image));
104             cameraView.setImage(frame);
105         });
106     }
107     modem.close();
108 }
109 catch (Exception e) {
110     // TODO: handle exception
111     e.printStackTrace();
112 }
113 }
114 };
115 Runnable packetRunnable = new Runnable() {
116
117     @Override
118     public void run() {
119         // TODO Auto-generated method stub
120         try {
121             //wait for all other threads to finish first
122             if(gpsThread != null) {
123                 if(gpsThread.isAlive()) gpsThread.join();
124             }
125             if(cameraThread != null) {
126                 if(cameraThread.isAlive()) cameraThread.join();
127             }
128             if(sessionTestThread != null) {
129                 if(sessionTestThread.isAlive()) sessionTestThread.join();
130             }
131             boolean oldcheckBoxStatus = errorPackets.isSelected();
132             int latencyBinCount = 10;
133             int[] latencyBins = new int[latencyBinCount];
134             int[] errorBins = new int[2];
135             Modem modem = new Modem(MODEM_BITRATE);
136             modem.setTimeout(MODEM_TIMEOUT);
137             getText(modem, null, "ithaki", DataTypes.INITIALIZATION);
138             packets = new ArrayList<>();
139             PacketTest packetTest = new PacketTest(modem);
140             while(packetsTab.isSelected()) {
141                 if(errorPackets.isSelected() != oldcheckBoxStatus) {
142                     packets.clear();
143                     oldcheckBoxStatus = errorPackets.isSelected();
144                     Platform.runLater(new Runnable() {
145
146                         @Override
147                         public void run() {
148                             // TODO Auto-generated method stub
149                             packetChart.getData().clear();
150                         }
151                     });
152                 }
153                 if(errorPackets.isSelected()) {
154                     packetTest.ackTest(1);
155                 }
156                 else {

```

```

157     packetTest.echoTest(1);
158 }
159 //Populate bar chart
160 Platform.runLater(new Runnable() {
161
162     @Override
163     public void run() {
164         // TODO Auto-generated method stub
165         XYChart.Series series = new XYChart.Series<>();
166         System.out.printf("Old packet total: %d ", packets.size());
167         packets.addAll(packetTest.packets);
168         System.out.printf("New packet total: %d added: %d\n",
169             ↪ packets.size(), packetTest.packets.size());
170         if(latencyDistribution) {
171             series.setName("Latency of packets in ms.");
172             for(int i = 0; i<latencyBinCount;++i) {
173                 latencyBins[i] = 0;
174             }
175             min = Integer.MAX_VALUE;
176             max = 0;
177             range = 0;
178             for(Packet packet: packets) {
179                 if(packet.latency > max) max = packet.latency;
180                 if(packet.latency < min) min = packet.latency;
181             }
182             range = max - min;
183             System.out.printf("MAX: %d MIN %d RANGE %d %n",
184                 ↪ max,min,range);
185             if(range > 0) {
186                 for(int i = 0; i<latencyBinCount;++i) {
187                     for(Packet packet: packets) {
188                         if(packet.latency >= min+ range*i/(double)
189                             ↪ latencyBinCount &&packet.latency
190                             ↪ <=min+
191                             ↪ range*(i+1)/(double)latencyBinCount) {
192                             latencyBins[i]++;
193                         }
194                     }
195                 }
196             }
197             else {
198                 for(Packet packet:packets) {
199                     latencyBins[0]++;
200                 }
201             }
202             for(int i = 0; i< latencyBinCount; ++i) {
203                 series.getData().add(new
204                     ↪ XYChart.Data(String.format("%.1f-%.1f",min +
205                     ↪ range*i/(double) latencyBinCount, min+
206                     ↪ range*(i+1)/(double) latencyBinCount),
207                     ↪ latencyBins[i]));
208             }
209         }
210     }
211     else {
212         series.setName("Packet error distribution");
213         errorBins[0] = 0;

```

```

204         errorBins[1] = 0;
205         for(Packet packet: packets) {
206             if(packet.wrong) {
207                 errorBins[1]++;
208             }
209             else {
210                 errorBins[0]++;
211             }
212             series.getData().add(new XYChart.Data("Correct",
213                 ↪ errorBins[0]));
214             series.getData().add(new XYChart.Data("Wrong",
215                 ↪ errorBins[1]));
216         }
217         //packetChart.getData().addAll(series);
218         if(packetChart.getData() != null)
219             ↪ packetChart.getData().clear();
220         packetChart.setData(FXCollections.observableArrayList(seri
221             ↪ es));
222         packetTest.packets.clear(); //empty packet test list to
223             ↪ get ready for new test
224     }
225     });
226     }
227     modem.close();
228 }
229 catch (Exception e) {
230     // TODO: handle exception
231     e.printStackTrace();
232 }
233 }
234 };
235 Runnable GPSRunnable = new Runnable() {
236
237     @Override
238     public void run() {
239         // TODO Auto-generated method stub
240         try {
241             //wait for all other threads to finish first
242             if(cameraThread != null) {
243                 if(cameraThread.isAlive()) cameraThread.join();
244             }
245             if(packetThread != null) {
246                 if(packetThread.isAlive()) packetThread.join();
247             }
248             if(sessionTestThread != null) {
249                 if(sessionTestThread.isAlive()) sessionTestThread.join();
250             }
251             Modem modem = new Modem(MODEM_BITRATE);
252             modem.setTimeout(MODEM_TIMEOUT);
253             getText(modem, null, "ithaki", DataTypes.INITIALIZATION);
254             String response =
255                 ↪ getText(modem,null,App.CODES.get("GPS")+"R=1100209",DataTypes.GPS);
256             System.out.println(response);
257             ArrayList<GPS> path = generatePathPoints(response);

```

```

254         StringBuilder mapRequest = new StringBuilder(App.CODES.get("GPS"));
255         for(GPS point: path) {
256             mapRequest.append(point.generateTParameter());
257         }
258         byte[] image = downloadImage(modem, true, mapRequest.toString());
259         Platform.runLater(new Runnable() {
260
261             @Override
262             public void run() {
263                 Image frame = new Image(new ByteArrayInputStream(image));
264                 // TODO Auto-generated method stub
265                 gpsView.setImage(frame);
266             }
267         });
268         modem.close();
269     }
270     catch (Exception e) {
271         // TODO: handle exception
272         e.printStackTrace();
273     }
274
275 }
276 };
277 tabPage.getSelectionModel().selectedItemProperty().addListener(new
↳ ChangeListener<Tab>() {
278
279     @Override
280     public void changed(ObservableValue<? extends Tab> observable, Tab oldValue, Tab
↳ newValue) {
281         // TODO Auto-generated method stub
282         System.out.printf("Tab changed from %s to %s!\n", oldValue.getId(),
↳ newValue.getId());
283         selectedTab = newValue;
284         //run time consuming tasks on separate thread so UI will not freeze
285         if(selectedTab.equals(cameraTab)) {
286             cameraThread = new Thread(cameraRunnable);
287             cameraThread.start();
288         }
289         else if(selectedTab.equals(gpsTab)) {
290             if(gpsView.getImage() == null) {
291                 gpsThread = new Thread(GPSRunnable);
292                 gpsThread.start();
293             }
294         }
295         else if(selectedTab.equals(packetsTab)) {
296             packetThread = new Thread(packetRunnable);
297             packetThread.start();
298         }
299         else if (selectedTab.equals(testTab)) {
300             }
301         }
302     });
303 }
304
305 @FXML
306 private void clearPacketData() {

```

```

307         if(packetChart.getData() != null) packetChart.getData().clear();
308         if(packets != null) packets.clear();
309     }
310
311     @FXML
312     private void enableLatency() {
313         latencyDistribution = true;
314     }
315
316     @FXML
317     private void enablePackErrors() {
318         latencyDistribution = false;
319     }
320
321     @FXML
322     private void cancelSessionTest() {
323         stopSessionTest = true;
324     }
325
326
327     @FXML
328     private void runTest() {
329         testTextArea.clear();
330         stopSessionTest = false;
331         sessionTestThread = new Thread(new Runnable() {
332             @Override
333             public void run() {
334                 // TODO Auto-generated method stub
335                 try {
336                     //wait for all other threads to finish first
337                     if(gpsThread != null) {
338                         if(gpsThread.isAlive()) gpsThread.join();
339                     }
340                     if(packetThread != null) {
341                         if(packetThread.isAlive()) packetThread.join();
342                     }
343                     if(cameraThread != null) {
344                         if(cameraThread.isAlive()) cameraThread.join();
345                     }
346                     Modem modem = new Modem(MODEM_BITRATE);
347                     modem.setTimeout(MODEM_TIMEOUT);
348                     getText(modem, testTextArea, "ithaki", DataTypes.INITIALIZATION);
349                     final String response = getText(modem, testTextArea, App.CODES.get("GPS")+"R",
350 ↪                      "1100209", DataTypes.GPS);
351                     System.out.println(response);
352                     Platform.runLater(new Runnable() {
353                         @Override
354                         public void run() {
355                             // TODO Auto-generated method stub
356                             testTextArea.appendText(response);
357                         }
358                     });
359                     ArrayList<GPS> path = generatePathPoints(response);
360                     StringBuilder mapRequest = new StringBuilder(App.CODES.get("GPS"));
361                     for(GPS point: path) {

```

```

362         mapRequest.append(point.generateTParameter());
363     }
364     File mapfile = new File("outputs/images/map.jpg");
365     downloadImage(modem,testTextArea,mapfile, true, mapRequest.toString());
366     File file;
367     try {
368         //modem.getInputStream().skip(modem.getInputStream().available());
369         for(int i = 0; i<imageCountSpinner.getValue();++i) { //frame test
370             if(stopSessionTest) break;
371             file = new File(String.format("outputs/images/image%d.jpg",i));
372             downloadImage(modem,testTextArea,file, true,null);
373         }
374         for(int i = 0; i<imageErrorCountSpinner.getValue();++i) {
375             if(stopSessionTest) break;
376             file = new
377                 ↪ File(String.format("outputs/images/imageError%d.jpg",i));
378             downloadImage(modem,testTextArea,file, false, null);
379         }
380     } catch (Exception e) {
381         // TODO Auto-generated catch block
382         e.printStackTrace();
383     }
384     PacketTest test = new PacketTest(modem,testTextArea);
385     test.ackTest(packetdurationSpinner.getValue());
386     test.toCSV("latenciesWithARQ.csv");
387     System.out.printf("BER is %.2f%n", test.getBER());
388     PacketTest test2 = new PacketTest(modem,testTextArea);
389     test2.echoTest(packetdurationSpinner.getValue());
390     test2.toCSV("latencies.csv");
391     modem.close();
392     Platform.runLater(new Runnable() {
393         @Override
394         public void run() {
395             // TODO Auto-generated method stub
396             Alert alert = new Alert(AlertType.INFORMATION, "Session test
397                 ↪ finished! Check outputs folder for images and CSVs!",
398                 ↪ ButtonType.OK);
399             alert.setTitle("Session test");
400             alert.showAndWait();
401         }
402     });
403 }
404 });
405 sessionTestThread.start();
406 }
407
408
409
410 protected synchronized byte[] downloadImage(Modem modem,boolean isErrorFree, String
411     ↪ mapRequest) {
412     int imageByte, previousImageByte = -1;
413     String command;
414     byte[] out;

```

```

414     if(mapRequest != null) {
415         command=mapRequest;
416         System.out.println("downloading GPS image!");
417     }
418     else {
419         if(isErrorFree) {
420             command = App.CODES.get("IMAGE_NO_ERROR");
421             System.out.println("downloading image without errors!");
422         }
423         else {
424             command = App.CODES.get("IMAGE_ERROR");
425             System.out.println("downloading image with errors!");
426         }
427     }
428
429     modem.setSpeed(80000);
430     modem.write((command+"\r").getBytes());
431     ArrayList<Byte> byteArrayList = new ArrayList<>();
432     try {
433         //FileOutputStream fileOutputStream = new FileOutputStream(file);
434         while(true) {
435             imageByte = modem.read();
436             if (imageByte == -1) break;
437             byteArrayList.add((byte)imageByte);
438             if (imageByte == 0xD9 && previousImageByte == 0xFF) break; //JPEG image
439             ↪ delimiter found
440             previousImageByte = imageByte;
441         }
442         modem.setSpeed(MODEM_BITRATE);
443     }
444     catch (Exception e) {
445         // TODO: handle exception
446         e.printStackTrace();
447     }
448     out = new byte[byteArrayList.size()];
449     for(int i = 0; i<byteArrayList.size();++i) {
450         out[i] = byteArrayList.get(i);
451     }
452
453     System.out.println("downloading image finished!");
454     return out;
455 }
456
457 protected synchronized static String getText(Modem modem,TextArea writeToTextArea, String
458 ↪ command,DataTypes datatype) {
459     StringBuilder stringBuilder = new StringBuilder();
460     int k;
461     if(datatype == DataTypes.INITIALIZATION) {
462         modem.open(command);
463     }
464     else {
465         modem.write((command+"\r").getBytes());
466     }
467     while(true) {
468         k = modem.read();
469         if(k==-1) break;

```

```

468     stringBuilder.append((char)k);
469     if(datatype == DataTypes.GPS && stringBuilder.indexOf("STOP ITHAKI GPS
↳ TRACKING\r\n") != -1) {
470         System.out.println("GPS end delimiter found!");
471         if(writeToTextArea != null) {
472             Platform.runLater(new Runnable() {
473                 @Override
474                 public void run() {
475                     // TODO Auto-generated method stub
476                     writeToTextArea.appendText("GPS end delimiter found!\n");
477                 }
478             });
479         }
480         break;
481     }
482     if(datatype == DataTypes.PACKETS && stringBuilder.indexOf("PSTOP") != -1) {
483         System.out.println("PACKET delimiter found!");
484         if(writeToTextArea != null) {
485             Platform.runLater(new Runnable() {
486                 @Override
487                 public void run() {
488                     // TODO Auto-generated method stub
489                     writeToTextArea.appendText("PACKET delimiter found!\n");
490                 }
491             });
492         }
493         break;
494     }
495     if(datatype == DataTypes.INITIALIZATION && stringBuilder.indexOf("the connection
↳ tested.\r\n\n") != -1) {
496         System.out.println("INITIALIZATION delimiter found!");
497         if(writeToTextArea != null) {
498             Platform.runLater(new Runnable() {
499                 @Override
500                 public void run() {
501                     // TODO Auto-generated method stub
502                     writeToTextArea.appendText("INITIALIZATION delimiter found!\n");
503                 }
504             });
505         }
506         break;
507     }
508 }
509 return stringBuilder.toString();
510 }
511
512
513 private ArrayList<GPS> generatePathPoints(String input){
514     ArrayList<GPS> path = new ArrayList<>();
515     BufferedReader reader = new BufferedReader(new StringReader(input));
516     String line;
517     try {
518         while((line = reader.readLine()) != null) {
519             if(line.startsWith("$GPGGA")) {
520                 GPS point = new GPS(line);
521                 path.add(point);

```

```

522     }
523 }
524 }
525 catch (IOException e) {
526     // TODO: handle exception
527     e.printStackTrace();
528 }
529 return path;
530 }
531
532 private static void downloadImage(Modem modem, TextArea writeToTextArea, File file, boolean
↪ isErrorFree, String mapRequest) {
533     int imageByte, previousImageByte = -1;
534     String command;
535     if(mapRequest != null) {
536         command=mapRequest;
537         System.out.println("downloading GPS image!");
538         if(writeToTextArea != null) {
539             Platform.runLater(new Runnable() {
540                 @Override
541                 public void run() {
542                     // TODO Auto-generated method stub
543                     writeToTextArea.appendText("downloading GPS image!\n");
544                 }
545             });
546         }
547     }
548     else {
549         if(isErrorFree) {
550             command = App.CODES.get("IMAGE_NO_ERROR");
551             System.out.println("downloading image WITHOUT errors!");
552             if(writeToTextArea != null) {
553                 Platform.runLater(new Runnable() {
554                     @Override
555                     public void run() {
556                         // TODO Auto-generated method stub
557                         writeToTextArea.appendText("Downloading image WITHOUT errors!\n");
558                     }
559                 });
560             }
561         }
562         else {
563             command = App.CODES.get("IMAGE_ERROR");
564             System.out.println("downloading image WITH errors!");
565             if(writeToTextArea != null) {
566                 Platform.runLater(new Runnable() {
567                     @Override
568                     public void run() {
569                         // TODO Auto-generated method stub
570                         writeToTextArea.appendText("downloading image WITH errors!\n");
571                     }
572                 });
573             }
574         }
575     }
576 }

```

```
577     modem.setSpeed(80000);
578     modem.write((command+"\r").getBytes());
579     ArrayList<Byte> bytelist = new ArrayList<>();
580     try {
581         FileOutputStream fileOutputStream = new FileOutputStream(file);
582         while(true) {
583             imageByte = modem.read();
584             if (imageByte == -1) break;
585             //fileOutputStream.write(imageByte);
586             bytelist.add((byte) imageByte);
587             if (imageByte == 0xD9 && previousImageByte == 0xFF) break; //JPEG image
588             ↪ delimiter found
589             previousImageByte = imageByte;
590         }
591         byte[] buffer = new byte[bytelist.size()];
592         for(int i = 0; i<bytelist.size();++i) {
593             buffer[i] = bytelist.get(i);
594         }
595         fileOutputStream.write(buffer);
596         fileOutputStream.flush();
597         fileOutputStream.close();
598         modem.setSpeed(MODEM_BITRATE);
599     }
600     catch (Exception e) {
601         // TODO: handle exception
602         e.printStackTrace();
603     }
604     System.out.println("downloading image finished!");
605     if(writeToTextArea != null) {
606         Platform.runLater(new Runnable() {
607             @Override
608             public void run() {
609                 // TODO Auto-generated method stub
610                 writeToTextArea.appendText("downloading image finished!\n");
611             }
612         });
613     }
614 }
615
```

2.3.4 Packet.java

```
1  package ithaki.diktya1;
2
3  class Packet{
4      boolean wrong;
5      int latency, repeats;
6
7      public Packet(int latency, boolean isWrong, int repeats) {
8          this.latency = latency;
9          this.repeats = repeats;
10         wrong = isWrong;
11     }
12 }
```

2.3.5 PacketTest.java

```
1  package ithaki.diktyai;
2
3  import java.io.BufferedWriter;
4  import java.io.File;
5  import java.io.FileWriter;
6  import java.io.IOException;
7  import java.util.ArrayList;
8
9  import ithakimodem.Modem;
10 import javafx.application.Platform;
11 import javafx.scene.control.TextArea;
12
13 class PacketTest {
14     protected ArrayList<Packet> packets = new ArrayList<>(); //Needed for CSV generation and
15     ↪ statistical processing.
16     private Modem modem;
17     private TextArea textArea;
18     private int latency;
19     private String data;
20
21     public PacketTest(Modem m) {
22         modem = m;
23         textArea = null;
24     }
25     public PacketTest(Modem m, TextArea textArea) {
26         modem = m;
27         this.textArea = textArea;
28     }
29     protected void echoTest(int seconds) {
30         long testStart = System.currentTimeMillis(),start,stop;
31         latency = 0;
32         while((System.currentTimeMillis()-testStart)/1000 < seconds &&
33             ↪ !GUIController.stopSessionTest) {
34             start = System.currentTimeMillis();
35             if(textArea != null) {
36                 data = GUIController.getText(modem,textArea,App.CODES.get("ECHO"),GUIControlle
37                 ↪ r.DataTypes.PACKETS);
38             }
39             else {
40                 data = GUIController.getText(modem,null,App.CODES.get("ECHO"),GUIController.Da
41                 ↪ taTypes.PACKETS);
42             }
43             stop = System.currentTimeMillis();
44             System.out.println(data);
45             latency = (int) (stop-start);
46             System.out.printf("Latency: %d ms.%n", latency);
47             if(textArea != null) {
48                 Platform.runLater(new Runnable() {
49                     @Override
50                     public void run() {
51                         // TODO Auto-generated method stub
52                         textArea.appendText(String.format("Latency: %d ms.%n", latency));
53                     }
54                 });
55             }
56         }
57     }
58 }
```

```

50     });
51     }
52     packets.add(new Packet(latency, false,0));
53 }
54 }
55
56
57 protected void ackTest(int seconds) {
58     long testStart = System.currentTimeMillis(),start,stop;
59     int repeatCount;
60     latency = 0;
61     boolean wrong;
62     while((System.currentTimeMillis()-testStart)/1000 < seconds &&
63     ↪ !GUIController.stopSessionTest) {
64         repeatCount = 0;
65         wrong = false;
66         start = System.currentTimeMillis();
67         if(textArea != null) {
68             data = GUIController.getText(modem,textArea,App.CODES.get("ACK"),GUIController
69             ↪ .DataTypes.PACKETS);
70         }
71         else {
72             data = GUIController.getText(modem,null,App.CODES.get("ACK"),GUIController.Dat
73             ↪ aTypes.PACKETS);
74         }
75         stop = System.currentTimeMillis();
76         System.out.println(data);
77         latency = (int) (stop-start);
78         while(!ARQChecksum(data)) {
79             repeatCount++;
80             System.out.printf("sequence %s is incorrect, sending NACK%n",data);
81             if(textArea != null) {
82                 Platform.runLater(new Runnable() {
83                     @Override
84                     public void run() {
85                         // TODO Auto-generated method stub
86                         textArea.appendText(String.format("sequence %s is incorrect,
87                         ↪ sending NACK%n",data));
88                     }
89                 });
90             }
91             start = System.currentTimeMillis();
92             data = GUIController.getText(modem,null,App.CODES.get("NACK"),GUIController.Da
93             ↪ taTypes.PACKETS);
94             stop = System.currentTimeMillis();
95             latency += (int) (stop-start);
96             wrong = true;
97         }
98     }
99     System.out.printf("Latency: %d ms.%n", latency);
100     if(textArea != null) {
101         Platform.runLater(new Runnable() {
102             @Override
103             public void run() {
104                 // TODO Auto-generated method stub
105                 textArea.appendText(String.format("%s%nLatency: %d ms.%n",
106                 ↪ data,latency));

```

```

100         }
101     });
102 }
103 packets.add(new Packet(latency, wrong,repeatCount));
104 }
105 }
106 protected double getBER() {
107     //assume on every NACK, all 16 characters are wrong. Packet has 58 bytes in total.
108     int wrongPacketCount = 0,totalPackets=0;
109     for(Packet packet:packets) {
110         totalPackets += packet.repeats +1;
111         if(packet.wrong) {
112             wrongPacketCount += packet.repeats;
113         }
114     }
115     return (wrongPacketCount/(double) totalPackets)*16/58;
116 }
117
118 private boolean ARQChecksum(String data) {
119     //TODO: implement ARQ algorithm.
120     try {
121         byte[] sequence = data.substring(31, 47).getBytes("ASCII");
122         final short fcs = Short.parseShort(data.substring(49,52));
123         short checksum = 0;
124         for(byte character: sequence) {
125             checksum ^= character;
126         }
127         if(checksum == fcs) {
128             return true;
129         }
130         else {
131             return false;
132         }
133     }
134     catch (Exception e) {
135         // TODO: handle exception
136         e.printStackTrace();
137         return false;
138     }
139 }
140 protected void toCSV(String filename) {
141     try {
142         File file = new File(String.format("outputs/csv/%s",filename));
143         BufferedWriter writer = new BufferedWriter(new FileWriter(file));
144         writer.write("packetNo,delay,isWrong,repeats\n");
145         Packet packet;
146         byte isPacketWrong = 0;
147         for(int i = 0;i<packets.size();++i) {
148             packet = packets.get(i);
149             isPacketWrong = (byte) (packet.wrong ? 1: 0);
150             writer.write(String.format("%d,%d,%d,%d\n",i+1,packet.latency,isPacketWrong,pa
151                 ↪ cket.repeats));
152         }
153         writer.flush();
154         writer.close();
155     }

```

```
155         catch (IOException e) {
156             // TODO: handle exception
157             e.printStackTrace();
158         }
159     }
160
161 }
162
```

2.3.6 GPS.java

```
1  package ithaki.diktya1;
2
3  import java.time.LocalDateTime;
4  import java.time.format.DateTimeFormatter;
5
6  class GPS{
7      //TODO: implement this class for GPS stuff and NMEA.
8      protected LocalDateTime time;
9      protected Coordinates latitude, longitude;
10     protected float elevation;
11     protected int satelliteCount; //Assumed that coordinates refer to N and E respectively.
12
13     public GPS(String positionData) {
14         String[] data = positionData.split(",");
15         time = LocalDateTime.parse(data[1], DateTimeFormatter.ofPattern("HHmmss.SSS"));
16         elevation = Float.parseFloat(data[9]);
17         satelliteCount = Integer.parseInt(data[7]);
18         latitude = new Coordinates(Double.parseDouble(data[2]));
19         longitude = new Coordinates(Double.parseDouble(data[4]));
20     }
21
22     @Override
23     public String toString() {
24         return String.format("t:%s %s N %s E, elev: %.1fm, sats:%d",
25             ↪ time,latitude,longitude,elevation,satelliteCount);
26     }
27
28     public String generateTParameter() {
29         return String.format("T=%d%d%d%d", longitude.degrees,longitude.minutes,longitude.seconds,
30             ↪ e.seconds,latitude.degrees,latitude.minutes,latitude.seconds);
31     }
32
33     class Coordinates{
34         public int degrees, minutes,seconds;
35         public Coordinates(double coord) {
36             degrees = (int) coord / 100;
37             minutes = (int) coord % 100;
38             seconds = (int) Math.round((coord % 100 - minutes)*60);
39         }
40         @Override
41         public String toString() {
42             return String.format("%d°%d'%d\"", degrees,minutes,seconds);
43         }
44     }
45 }
```

43 }
44

2.4 Αρχείο GUI

2.4.1 ithaki.fxml

```

1  <?xml version="1.0" encoding="UTF-8"?>
2
3  <?import javafx.geometry.Insets?>
4  <?import javafx.scene.chart.BarChart?>
5  <?import javafx.scene.chart.CategoryAxis?>
6  <?import javafx.scene.chart.NumberAxis?>
7  <?import javafx.scene.control.Button?>
8  <?import javafx.scene.control.CheckBox?>
9  <?import javafx.scene.control.Label?>
10 <?import javafx.scene.control.RadioButton?>
11 <?import javafx.scene.control.Spinner?>
12 <?import javafx.scene.control.Tab?>
13 <?import javafx.scene.control.TabPane?>
14 <?import javafx.scene.control.TextArea?>
15 <?import javafx.scene.control.ToggleGroup?>
16 <?import javafx.scene.image.ImageView?>
17 <?import javafx.scene.layout.AnchorPane?>
18 <?import javafx.scene.layout.ColumnConstraints?>
19 <?import javafx.scene.layout.GridPane?>
20 <?import javafx.scene.layout.RowConstraints?>
21 <?import javafx.scene.layout.VBox?>
22 <?import javafx.scene.text.Font?>
23
24 <TabPane fx:id="tabPane" maxHeight="-Infinity" maxWidth="-Infinity" minHeight="-Infinity"
  ↳ minWidth="-Infinity" prefHeight="400.0" prefWidth="600.0" tabClosingPolicy="UNAVAILABLE"
  ↳ xmlns="http://javafx.com/javafx/19" xmlns:fx="http://javafx.com/fxml/1"
  ↳ fx:controller="ithaki.diktya1.GUIController">
25   <tabs>
26     <Tab fx:id="sessTab" closable="false" text="Session details">
27       <content>
28         <AnchorPane minHeight="0.0" minWidth="0.0" prefHeight="180.0" prefWidth="200.0">
29           <children>
30             <GridPane prefHeight="372.3130558283617" prefWidth="600.2087155618656"
              ↳ AnchorPane.bottomAnchor="0.0" AnchorPane.leftAnchor="0.0"
              ↳ AnchorPane.rightAnchor="0.0" AnchorPane.topAnchor="0.0">
31               <columnConstraints>
32                 <ColumnConstraints hgrow="SOMETIMES" maxWidth="453.28697155784846"
                  ↳ minWidth="10.0" prefWidth="180.31302077641374" />
33                 <ColumnConstraints hgrow="SOMETIMES" maxWidth="272.9739507814347"
                  ↳ minWidth="0.0" prefWidth="272.9739507814347" />
34               </columnConstraints>
35               <rowConstraints>
36                 <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />
37                 <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />
38                 <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />
39                 <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />
40                 <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />
41                 <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />

```

```

42     </rowConstraints>
43     <children>
44         <Label prefHeight="28.0" prefWidth="260.0" text="ACK code:"
45             ↳ GridPane.rowIndex="4">
46             <font>
47                 <Font size="19.0" />
48             </font>
49         </Label>
50         <Label text="Image request code (no errors):" GridPane.rowIndex="1">
51             <font>
52                 <Font size="19.0" />
53             </font>
54         </Label>
55         <Label prefHeight="28.0" prefWidth="261.0" text="Echo request code:"
56             <font>
57                 <Font size="19.0" />
58             </font>
59         </Label>
60         <Label prefHeight="28.0" prefWidth="260.0" text="Image request code
61             ↳ (errors):" GridPane.rowIndex="2">
62             <font>
63                 <Font size="19.0" />
64             </font>
65         </Label>
66         <Label prefHeight="28.0" prefWidth="260.0" text="GPS code:"
67             ↳ GridPane.rowIndex="3">
68             <font>
69                 <Font size="19.0" />
70             </font>
71         </Label>
72         <Label text="NACK code:" GridPane.rowIndex="5">
73             <font>
74                 <Font size="19.0" />
75             </font>
76         </Label>
77         <Label fx:id="echoLabel" text="Label" textFill="#0016de"
78             ↳ GridPane.columnIndex="1">
79             <font>
80                 <Font size="19.0" />
81             </font>
82         </Label>
83         <Label fx:id="imageLabel" text="Label" textFill="#0016de"
84             ↳ GridPane.columnIndex="1" GridPane.rowIndex="1">
85             <font>
86                 <Font size="19.0" />
87             </font>
88         </Label>
89         <Label fx:id="imageErrorLabel" text="Label" textFill="#0016de"
90             ↳ GridPane.columnIndex="1" GridPane.rowIndex="2">
91             <font>

```

```

91         <Font size="19.0" />
92     </font>
93 </Label>
94 <Label fx:id="ackLabel" text="Label" textFill="#0016de"
95     ↳ GridPane.columnIndex="1" GridPane.rowIndex="4">
96     <font>
97         <Font size="19.0" />
98     </font>
99 </Label>
100 <Label fx:id="nackLabel" text="Label" textFill="#0016de"
101     ↳ GridPane.columnIndex="1" GridPane.rowIndex="5">
102     <font>
103         <Font size="19.0" />
104     </font>
105 </Label>
106 </children>
107 </GridPane>
108 </children></AnchorPane>
109 </content>
110 </Tab>
111 <Tab fx:id="cameraTab" closable="false" text="Camera">
112     <content>
113         <AnchorPane minHeight="0.0" minWidth="0.0" prefHeight="180.0" prefWidth="200.0">
114             <children>
115                 <VBox alignment="CENTER" layoutX="95.0" layoutY="50.0"
116                     ↳ prefHeight="372.3130558283617" prefWidth="600.2087155618656"
117                     ↳ AnchorPane.bottomAnchor="0.0" AnchorPane.leftAnchor="0.0"
118                     ↳ AnchorPane.rightAnchor="0.0" AnchorPane.topAnchor="0.0">
119                     <children>
120                         <Label text="Image is loading, please wait! Download rate is 80 kbps.">
121                             <font>
122                                 <Font size="12.0" />
123                             </font>
124                         </Label>
125                         <ImageView fx:id="cameraView" fitHeight="299.0" fitWidth="474.0"
126                             ↳ pickOnBounds="true" preserveRatio="true" />
127                     </children>
128                 </VBox>
129             </children></AnchorPane>
130 </content>
131 </Tab>
132 <Tab fx:id="gpsTab" closable="false" text="GPS">
133     <content>
134         <AnchorPane minHeight="0.0" minWidth="0.0" prefHeight="180.0" prefWidth="200.0">
135             <children>
136                 <VBox alignment="CENTER" layoutX="145.0" layoutY="86.0"
137                     ↳ prefHeight="372.3130558283617" prefWidth="600.2087155618656"
138                     ↳ AnchorPane.bottomAnchor="0.0" AnchorPane.leftAnchor="0.0"
139                     ↳ AnchorPane.rightAnchor="0.0" AnchorPane.topAnchor="0.0">
140                     <children>
141                         <Label text="Image is loading, please wait! Download rate is 80 kbps.">
142                             <font>
143                                 <Font size="12.0" />
144                             </font>
145                         </Label>
146                         <ImageView fx:id="gpsView" fitHeight="299.0" fitWidth="494.0"
147                             ↳ pickOnBounds="true" preserveRatio="true" />

```

```

138         </children>
139     </VBox>
140 </children></AnchorPane>
141 </content>
142 </Tab>
143 <Tab fx:id="packetsTab" closable="false" text="Packets">
144     <content>
145         <AnchorPane minHeight="0.0" minWidth="0.0" prefHeight="180.0" prefWidth="200.0">
146             <children>
147                 <GridPane layoutX="159.0" layoutY="103.0" prefHeight="372.3130558283617"
148                     ↪ prefWidth="600.2087155618656" AnchorPane.bottomAnchor="0.0"
149                     ↪ AnchorPane.leftAnchor="0.0" AnchorPane.rightAnchor="0.0"
150                     ↪ AnchorPane.topAnchor="0.0">
151                     <columnConstraints>
152                         <ColumnConstraints hgrow="SOMETIMES" maxWidth="414.5390897062191"
153                             ↪ minWidth="10.0" prefWidth="414.5390897062191" />
154                         <ColumnConstraints hgrow="SOMETIMES" maxWidth="294.3304715421566"
155                             ↪ minWidth="10.0" prefWidth="185.8087430265316" />
156                     </columnConstraints>
157                     <rowConstraints>
158                         <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />
159                     </rowConstraints>
160                     <children>
161                         <BarChart fx:id="packetChart" title="Packets">
162                             <xAxis>
163                                 <CategoryAxis side="BOTTOM" tickLabelRotation="30.0" />
164                             </xAxis>
165                             <yAxis>
166                                 <NumberAxis side="LEFT" />
167                             </yAxis>
168                         </BarChart>
169                         <VBox alignment="CENTER_LEFT" prefHeight="200.0" prefWidth="100.0"
170                             ↪ spacing="10.0" GridPane.columnIndex="1">
171                             <children>
172                                 <CheckBox fx:id="errorPackets" mnemonicParsing="false"
173                                     ↪ text="Packets with errors">
174                                     <font>
175                                         <Font size="14.0" />
176                                     </font>
177                                 </CheckBox>
178                                 <RadioButton fx:id="latencyDist" contentDisplay="TOP"
179                                     ↪ mnemonicParsing="false" onAction="#enableLatency"
180                                     ↪ prefHeight="18.0" prefWidth="184.0" selected="true"
181                                     ↪ text="Latency distribution">
182                                     <font>
183                                         <Font size="14.0" />
184                                     </font>
185                                     <toggleGroup>
186                                         <ToggleGroup fx:id="packetToggleGroup" />
187                                     </toggleGroup>
188                                 </RadioButton>
189                                 <RadioButton fx:id="packerrDist" mnemonicParsing="false"
190                                     ↪ onAction="#enablePackErrors" text="Packet error
191                                     ↪ distribution" toggleGroup="$packetToggleGroup">
192                                     <font>
193                                         <Font size="14.0" />
194                                     </font>

```

```

182         </font>
183     </RadioButton>
184     <Button mnemonicParsing="false" onAction="#clearPacketData"
185     ↪ prefHeight="28.0" prefWidth="175.0" text="Clear data!">
186         <font>
187             <Font size="12.0" />
188         </font>
189         <VBox.margin>
190             <Insets left="5.0" />
191         </VBox.margin>
192     </Button>
193 </children>
194 <GridPane.margin>
195     <Insets top="5.0" />
196 </GridPane.margin>
197 </VBox>
198 </children>
199 </GridPane>
200 </children></AnchorPane>
201 </content>
202 </Tab>
203 <Tab fx:id="testTab" closable="false" text="Session Test">
204     <content>
205         <AnchorPane minHeight="0.0" minWidth="0.0" prefHeight="180.0" prefWidth="200.0">
206             <children>
207                 <GridPane prefHeight="372.3130558283617" prefWidth="600.2087155618656"
208                 ↪ AnchorPane.bottomAnchor="0.0" AnchorPane.leftAnchor="0.0"
209                 ↪ AnchorPane.rightAnchor="0.0" AnchorPane.topAnchor="0.0">
210                     <columnConstraints>
211                         <ColumnConstraints hgrow="SOMETIMES" maxWidth="428.73043492106285"
212                         ↪ minWidth="10.0" prefWidth="367.9739103297795" />
213                         <ColumnConstraints hgrow="SOMETIMES" maxWidth="291.82613804606285"
214                         ↪ minWidth="10.0" prefWidth="231.904370504859" />
215                     </columnConstraints>
216                     <rowConstraints>
217                         <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />
218                     </rowConstraints>
219                     <children>
220                         <VBox alignment="CENTER" prefHeight="200.0" prefWidth="100.0"
221                         ↪ GridPane.columnIndex="1">
222                             <children>
223                                 <GridPane>
224                                     <columnConstraints>
225                                         <ColumnConstraints hgrow="SOMETIMES"
226                                         ↪ maxWidth="151.76525974208795" minWidth="10.0"
227                                         ↪ prefWidth="151.76525974208795" />
228                                         <ColumnConstraints hgrow="SOMETIMES"
229                                         ↪ maxWidth="112.0000272062191" minWidth="10.0"
230                                         ↪ prefWidth="79.97387844612118" />
231                                     </columnConstraints>
232                                     <rowConstraints>
233                                         <RowConstraints minHeight="10.0" prefHeight="30.0"
234                                         ↪ vgrow="SOMETIMES" />
235                                         <RowConstraints minHeight="10.0" prefHeight="30.0"
236                                         ↪ vgrow="SOMETIMES" />
237                                         <RowConstraints minHeight="10.0" prefHeight="30.0"
238                                         ↪ vgrow="SOMETIMES" />
239                                     </rowConstraints>

```

```

226         </rowConstraints>
227         <children>
228             <Label prefHeight="18.0" prefWidth="134.0" text="Packet
↳ test duration (sec)">
229                 <font>
230                     <Font size="12.0" />
231                 </font>
232             </Label>
233             <Spinner fx:id="packetdurationSpinner" editable="true"
↳ GridPane.columnIndex="1" />
234             <Label text="Image without errors count"
↳ GridPane.rowIndex="1">
235                 <font>
236                     <Font size="12.0" />
237                 </font>
238             </Label>
239             <Spinner fx:id="imageCountSpinner" editable="true"
↳ GridPane.columnIndex="1" GridPane.rowIndex="1" />
240             <Spinner fx:id="imageErrorCountSpinner" editable="true"
↳ GridPane.columnIndex="1" GridPane.rowIndex="2" />
241             <Label text="Image with errors count"
↳ GridPane.rowIndex="2">
242                 <font>
243                     <Font size="12.0" />
244                 </font>
245             </Label>
246         </children>
247         <VBox.margin>
248             <Insets bottom="15.0" />
249         </VBox.margin>
250         <padding>
251             <Insets left="5.0" />
252         </padding>
253     </GridPane>
254     <Button fx:id="sessionTestBtn" mnemonicParsing="false"
↳ onAction="#runTest" prefHeight="28.0" prefWidth="199.0"
↳ text="Run test">
255         <VBox.margin>
256             <Insets bottom="10.0" />
257         </VBox.margin></Button>
258     <Button fx:id="sessionTestBtn1" mnemonicParsing="false"
↳ onAction="#cancelSessionTest" prefHeight="28.0"
↳ prefWidth="199.0" text="Cancel test" />
259 </children>
260 </VBox>
261 <TextArea fx:id="testTextArea" editable="false" prefHeight="372.0"
↳ prefWidth="376.0">
262     <font>
263         <Font size="12.0" />
264     </font>
265 </TextArea>
266 </children>
267 </GridPane>
268 </children>
269 </AnchorPane>
270 </content>

```

```
271     </Tab>
272 </tabs>
273 </TabPane>
274
```

2.5 Αρχείο διαχείρισης βιβλιοθηκών Maven

```
1  <project xmlns="http://maven.apache.org/POM/4.0.0"
   ↪  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
   ↪  http://maven.apache.org/maven-v4_0_0.xsd">
3    <modelVersion>4.0.0</modelVersion>
4    <groupId>ithaki</groupId>
5    <artifactId>diktya1</artifactId>
6    <version>1.0.0-BETA</version>
7    <properties>
8      <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
9      <maven.compiler.source>11</maven.compiler.source>
10     <maven.compiler.target>11</maven.compiler.target>
11  </properties>
12  <dependencies>
13    <dependency>
14      <groupId>org.openjfx</groupId>
15      <artifactId>javafx-controls</artifactId>
16      <version>17</version>
17    </dependency>
18    <dependency>
19      <groupId>org.openjfx</groupId>
20      <artifactId>javafx-fxml</artifactId>
21      <version>17</version>
22    </dependency>
23    <dependency>
24      <groupId>org.jsoup</groupId>
25      <artifactId>jsoup</artifactId>
26      <version>1.15.4</version>
27    </dependency>
28    <dependency>
29      <groupId>com.ithaki</groupId>
30      <artifactId>ithakimodem</artifactId>
31      <version>1.0</version>
32    </dependency>
33  </dependencies>
34  <build>
35    <plugins>
36      <plugin>
37        <groupId>org.apache.maven.plugins</groupId>
38        <artifactId>maven-compiler-plugin</artifactId>
39        <version>3.8.0</version>
40        <configuration>
41          <release>11</release>
42        </configuration>
43      </plugin>
44      <plugin>
45        <groupId>org.apache.maven.plugins</groupId>
46        <artifactId>maven-install-plugin</artifactId>
47        <version>3.1.0</version>
```

```

48         <executions>
49             <execution>
50                 <phase>clean</phase>
51                 <goals>
52                     <goal>install-file</goal>
53                 </goals>
54                 <configuration>
55                     <file>lib/ithakimodem.jar</file>
56                     <groupId>com.ithaki</groupId>
57                     <artifactId>ithakimodem</artifactId>
58                     <version>1.0</version>
59                     <packaging>jar</packaging>
60                 </configuration>
61             </execution>
62         </executions>
63     </plugin>
64     <plugin>
65         <groupId>org.openjfx</groupId>
66         <artifactId>javafx-maven-plugin</artifactId>
67         <version>0.0.6</version>
68         <executions>
69             <execution>
70                 <!-- Default configuration for running -->
71                 <!-- Usage: mvn clean javafx:run -->
72                 <id>default-cli</id>
73                 <configuration>
74                     <mainClass>ithaki.diktya1.App</mainClass>
75                 </configuration>
76             </execution>
77         </executions>
78     </plugin>
79
80
81     <plugin>
82         <artifactId>maven-shade-plugin</artifactId>
83         <executions>
84             <execution>
85                 <phase>package</phase>
86                 <goals>
87                     <goal>shade</goal>
88                 </goals>
89                 <configuration>
90                     <transformers>
91                         <transformer implementation="org.apache.maven.plugins.shade.re_j
92                             ↪ source.ManifestResourceTransformer">
93                             <mainClass>ithaki.diktya1.userApplication</mainClass>
94                         </transformer>
95                     </transformers>
96                 </configuration>
97             </execution>
98         </executions>
99     </plugin>
100 </plugins>
101 </build>
102 </project>

```